

Entornos de desarrollo

UNIDAD 3

3. Diseño y realización de pruebas.....

Diseño y realización de pruebas

Objetivos

- Comprender la filosofía de las pruebas del software.
- Emplear las estrategias adecuadas para la realización de pruebas a lo largo del ciclo de vida del software.
- Conocer y aplicar técnicas de diseño de casos de prueba de caja blanca y de caja negra.
- Conocer de qué manera se documentan las pruebas.
- Utilizar herramientas de depuración en un entorno de desarrollo.
- Realizar pruebas automáticas en un entorno de desarrollo.

Contenidos

- 3.1. Filosofía de las pruebas del software
- 3.2. Estrategia de pruebas del software
- 3.3. Técnicas de diseño de casos de prueba
- 3.4. Documentación de las pruebas
- 3.5. Herramientas de depuración
- 3.6. Pruebas automáticas

Introducción

Cuando un equipo de desarrollo acepta el encargo de crear una aplicación Informática, desea que el resultado de su trabajo sea de calidad, esto es, que la aplicación responda a las necesidades del cliente y que tenga el menor número posible de errores. El objetivo de la etapa de pruebas, posterior a la programación, es descubrir dichos defectos antes de que el software sea entregado al cliente.

En esta unidad, se explica cómo abordar esta importante tarea y las diferentes técnicas de diseño de casos de prueba. También se verá de qué manera un entorno de desarrollo puede asistir a las personas que desarrollan programas Informáticos en las tareas de depuración o corrección de errores encontrados en esta fase. Para finalizar, se describe el empleo de una herramienta que sirve para la realización de pruebas unitarias de manera automatizada.

***Una aplicación con errores
NO es una aplicación de calidad***

3.1. Filosofía de las pruebas del software

Las pruebas constituyen una de las tareas del ciclo de vida del software y tienen como objetivo fundamental la detección de defectos que se han cometido inadvertidamente durante el proceso de construcción del software.

*Una aplicación con cero errores
raramente se consigue.
Se trata de alcanzar un **EQUILIBRIO**
entre el tiempo y el esfuerzo
de la realización de las pruebas y
la inexistencia de errores.
“Ningún programador es infalible”*

El objetivo de las pruebas puede percibirse desde dos puntos de vista:

1. Mediante las pruebas se pretende, por un lado, comprobar si se está construyendo el producto correcto, es decir, el que desea el cliente, lo cual se relaciona con el concepto de **validación**.
2. Por otro lado, se trata de comprobar si el producto que se está desarrollando funciona correctamente, es decir, si lo que hace lo realiza de manera correcta. Esto se refiere al concepto de **verificación**.

Se trata de ver:

1. *Que el programa es lo que el cliente necesita*
2. *Que el programa funciona de forma correcta*

Recuerda



El hecho de que se encuentren errores a lo largo de la fase de pruebas de una aplicación no debe entenderse como un fracaso del trabajo previo realizado. Al contrario, se debe considerar como algo positivo el hecho de que se encuentren errores después de terminada la programación, pues esto permite eliminarlos antes de entregar la aplicación y evita que estos sean descubiertos por el cliente. No se debe olvidar que «todos cometemos errores» y, como dice el dicho popular, «rectificar es de sabios».

■ 3.2. Estrategia de pruebas del software

Las pruebas siempre comienzan por los componentes más pequeños, es decir, se empieza probando pequeñas porciones de software y se va incrementando de manera progresiva el alcance de la prueba. Se suelen llevar a cabo cuatro tipos de pruebas secuenciales que se exponen en los siguientes subapartados.

■ ■ 3.2.1. Prueba de unidad

■ ■ 3.2.2. Prueba de integración

■ ■ 3.2.3. Prueba del sistema

■ ■ 3.2.4. Prueba de validación

3.2.1. Prueba de unidad

Se comienza probando módulos individuales

3.2.2. Prueba de integración

El objetivo de estas pruebas, que se realizan a continuación de las pruebas de unidad, es comprobar si las clases de las que consta el software funcionan correctamente cuando cooperan entre ellas con el objetivo de realizar las tareas encomendadas al programa.

*Las
iremos
viendo*

1. Prueba basada en hebra: Integra el conjunto de clases requeridas para responder a una entrada o evento del sistema. Cada hebra se integra y prueba de manera individual.
2. Prueba basada en uso: se comienza probando las clases independientes, esto es, aquellas que usan muy pocas clases de servidor o ninguna. Después de probar estas, se examina la siguiente capa de clases, llamadas clases dependientes, que son utilizadas por las primeras. Esta secuencia de pruebas para las capas de clases dependientes continúa hasta que se construye todo el sistema.

3.2.3. Prueba del sistema

- Las iremos viendo todos**
- **Pruebas de recuperación**, ante tolerancia a fallos.
 - **Pruebas de seguridad**, ante jaqueo externo o de protección de datos.
 - **Pruebas de esfuerzo**, ante saturación del sistema por saturación de recursos.
 - **Pruebas de rendimiento**, en tiempos adecuados y de forma eficiente.
 - **Pruebas de despliegue**, en diferentes plataformas y sistemas operativos.

3.2.4. Prueba de validación

El objetivo de la prueba de validación, también llamada **prueba de aceptación**, es determinar si el software es considerado **válido por parte la persona usuaria** y si está **preparado para su implantación** en el entorno de las personas que lo vayan a usar.

En este tipo de pruebas, **participan activamente aquellas personas que lo van a utilizar**, las cuales, **ayudadas por el equipo de pruebas**, deben ejecutar las pruebas. Esta es la fase final del proceso software, en la que la persona usuaria da su consentimiento para su uso o explotación.

Debe estar acorde con la Especificación de Requisitos de Software (ERS) y es la FASE FINAL.

Esquema de las pruebas del software:



Figura 3.1. Representación del modelo en V, en el que la fase posterior a la programación (las pruebas) se divide en varias subfases que corresponden a las etapas del ciclo de vida en cascada, si bien las etapas de análisis y diseño se dividen cada una de ellas en dos.

■ 3.3. Técnicas de diseño de casos de prueba

1. **Pruebas de caja blanca o pruebas estructurales:** examinan los detalles de cada módulo, para lo que se debe disponer del código fuente. A través de dicho código, se prueban los diferentes caminos, los bucles, las variables, etcétera.
2. **Pruebas de caja negra o pruebas funcionales:** se considera al software como una caja negra que recibe una serie de entradas y proporciona una serie de salidas. El objetivo de estas pruebas es validar los requisitos funcionales.

ACTIVIDAD DICCIONARIO:

Elabora un diccionario con los siguientes términos:

- VALIDACIÓN DE LA APLICACIÓN
- ACEPTACIÓN DE LA APLICACIÓN
- PRUEBA DE UNIDAD
- PRUEBA DE INTEGRACIÓN
- PRUEBA DEL SISTEMA
- PRUEBA DE VALIDACIÓN
- PRUEBA DE ACEPTACIÓN
- ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE (ERS)
- PRUEBAS DE CAJA BLANCA
- PRUEBAS ESTRUCTURALES
- PRUEBAS DE CAJA NEGRA
- PRUEBAS FUNCIONALES

ACTIVIDAD TIPO TEST:

Haz con tu compañero un test de 10 preguntas de tipo test.

3.3.1. Pruebas estructurales o de caja blanca

El objetivo de las pruebas de caja blanca es crear casos de prueba que permitan revisar detalles internos del software. Así, de estas pruebas se derivan casos de prueba que:

1. Garantizan que todas las rutas independientes dentro de un módulo se revisaron al menos una vez.
2. Revisan todas las decisiones lógicas en sus lados verdadero y falso.
3. Ejecutan todos los bucles en sus fronteras y dentro de sus fronteras operativas.
4. Revisan estructuras de datos internas para garantizar su validez.

Existen diferentes técnicas de diseño de casos de prueba de caja blanca, que se exponen seguidamente.



Prueba del camino básico

Esta técnica también recibe el nombre de **prueba de ruta básica** y fue propuesta por Thomas J. McCabe. Esta técnica permite generar una métrica de la complejidad del módulo probado llamada **complejidad ciclomática de McCabe**, que es un número entero que indica el número de caminos independientes que hay en el código y, en consecuencia, el número de casos de prueba que hay que ejecutar para garantizar que se recorren todas las sentencias del código al menos una vez. Para calcular esta métrica y, en consecuencia, generar los casos de prueba, es necesario representar el código fuente que se va a probar en un **grafo de flujo**.

Para crear un grafo de flujo, se deben llevar a cabo los siguientes pasos:

1. Se asigna un número único a cada sentencia del código y a cada condición. No obstante, si se dispone de varias instrucciones en secuencia, es posible asignar un número único a todas ellas y estas serán tratadas como un bloque. Cada uno de estos elementos (sentencia o grupo de sentencias en secuencia o condición) constituirá un **nodo** del grafo de flujo, que se representará mediante un círculo en cuyo interior aparecerá el número asignado.
2. Se unen los nodos mediante flechas llamadas **aristas** que representan el flujo de control y son similares a las flechas de un diagrama de flujo. Se deben representar las estructuras básicas de programación tal y como se muestra en Figura 3.2.

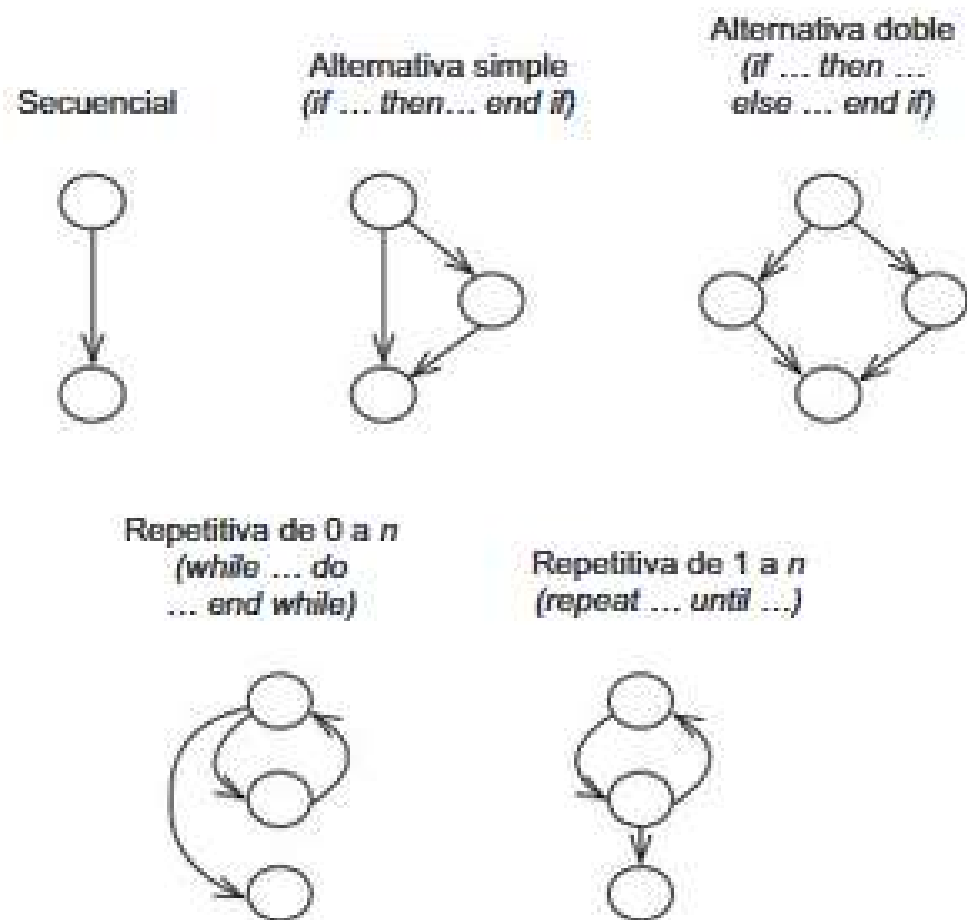


Figura 3.2. Representación de los grafos de flujo para diferentes estructuras de control.

Una vez creado el grafo de flujo, se puede calcular la complejidad ciclomática de McCabe $V(G)$ de cualquiera de las tres formas siguientes:

- **Numero de regiones del grafo** considerando una región como una zona del grafo cerrada. No obstante, también se debe considerar como región la región externa (no cerrada).

- De acuerdo con la fórmula:

$$V(G) = A - N + 2$$

donde **A es el numero de aristas** del grafo, y N , el número de nodos.

- Según la ecuación:

$$V(G) = NP + 1$$

donde **NP es el numero de nodos predicados** del grafo de flujo, es decir, el número de nodos que contienen condiciones.

SE VE MEJOR CON LA
ACTIVIDAD RESUELTA

El **valor $V(G)$ indica el número de caminos independientes del grafo** y, por tanto, el número de **casos de prueba que hay que generar** para garantizar que se han recorrido todas las instrucciones y condiciones del código.

El último paso de este método es generar casos de prueba que recorran cada uno de los caminos, para lo que se deben elegir los datos de entrada que fuercen cada uno de esos caminos. Puede ocurrir que alguno de los caminos no pueda recorrerse de manera independiente, sino como parte de otro camino.

Para cada caso de prueba, se deben comparar los resultados obtenidos con los esperados. La no coincidencia indicará la existencia de algún defecto y, en consecuencia, se debe llevar a cabo un proceso de depuración.

Actividad resuelta 3.1

Prueba del camino básico

En este ejercicio se aplica la prueba del camino básico a un método que lee una serie de números hasta que se introduce un cero y muestra la media de los números positivos introducidos y la media de los números negativos.

Solución

La primera tarea que se debe a llevar a cabo es asignar un número entero a cada secuencia de instrucciones y a cada condición:

```
begin
    1  int num = 0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
    float media_pos=0, media_neg=0;
    System.out.print("Introduce número: ");
    num=Entrada.entero();
    2  while (num != 0)
    3      if (num > 0)
    4          cont_pos++;
          suma_pos+=num;
    5      else
          cont_neg++;
          suma_neg+=num;
    6      endif;
      System.out.print("Introduce número: ");
      num=Entrada.entero();
    7  endwhile;
    8  if (cont_pos != 0)
    9      media_pos = (float)suma_pos/cont_pos;
      System.out.println("Media de los positivos: "+ media_pos);
    10 endif;
    11 if (cont_neg != 0)
    12     media_neg= (float)suma_neg/cont_neg;
      System.out.println("Media de los negativos: "+ media_neg);
    13 endif;
end;
```

Es posible incluir dentro un mismo nodo varias instrucciones en secuencia, como es el caso de las instrucciones incluidas dentro del nodo 1. También se puede incluir, dentro del mismo nodo, la instrucción que indica el fin de una instrucción condicional y la instrucción o las instrucciones siguientes en secuencia, como se ha hecho para el nodo 6. Asimismo, es posible incluir, dentro de un mismo nodo, la instrucción que indica el fin de una estructura repetitiva junto con las instrucciones anteriores dentro del bucle si son instrucciones en secuencia. Por este motivo, aunque no se ha hecho, también se podrían haber incluido en el mismo nodo las instrucciones asignadas a los nodos 6 y 7.

El siguiente paso es construir el grafo de flujo de acuerdo con las estructuras mostradas en la Figura 3.2. El grafo de flujo quedaría como el que se muestra en la Figura 3.3. En el grafo, se han marcado en rojo los nodos predicados y se ha asignado un número a cada región en color azul.



Una vez que se tiene el grafo de flujo, ya se puede calcular la complejidad ciclomática, $V(G)$, de las tres formas que se ha indicado antes:

$V(G) = \text{número de regiones} = 5$ **$A = \text{nº de flechas}$**

$V(G) = A - N + 2 = 16 - 13 + 2 = 5$ **$N = \text{nº de nodos}$**

$V(G) = NP + 1 = 4 + 1 = 5$ **$NP = \text{nº de nodos}$**

Como $V(G)$ toma el valor 5, se deduce que hay cinco caminos independientes en el código examinado y, por tanto, el número de casos de prueba que hay que crear para cubrir todas las instrucciones y condiciones del programa es 5. La siguiente tarea es encontrar esos cinco caminos independientes. Para ello, hay que tener en cuenta que cada camino independiente incluye una ruta nueva en relación con los anteriores:

Camino 1: 1-2-8-10-11-13

Camino 2: 1-2-3-4-6-7-2-8-10-11-13

Camino 3: 1-2-3-4-6-7-2-8-9-10-11-13

Camino 4: 1-2-3-5-6-7-2-8-10-11-13

Camino 5: 1-2-3-5-6-7-8-10-11-12-13

Con objeto de probar el camino 1, se debe introducir como primer número el 0 con el fin de que no se entre en el bucle ni una sola vez y se pase directamente del nodo 2 al nodo 8. En este caso, el programa no debe mostrar nada, ya que no se ha introducido ningún número ni positivo, ni negativo.

El camino 2 no se puede probar como tal, ya que al introducir un número positivo (para pasar por el nodo 4), se debería pasar por el nodo 9, que muestra la media de los números positivos. Por consiguiente, este camino debe probarse como parte de otro.

Para probar el camino 3, hay que suministrar al programa, en primer lugar, un número positivo (para pasar por el nodo 4), por ejemplo, el número 8, y un 0 a continuación (para no pasar más por el bucle). El programa deberá indicar que la media de los números positivos es 8. Al probar el camino 3, ya se prueba el camino 2 como parte de él.

El camino 4 tampoco se puede probar como tal, pues al introducir un número negativo (para pasar por el nodo 5), se debe pasar por el nodo 12 para mostrar la media de los números negativos.

Por último, a fin de probar el camino 5, se debe suministrar al programa un número negativo (para pasar por el nodo 5), por ejemplo, el -4, y un 0 a continuación. El programa deberá indicar que la media de los números negativos es -4. Al probar el camino 5, se prueba también el camino 4 como parte de él.

Para cada caso de prueba, se comprueba que el resultado mostrado coincide con el esperado, de forma que la no coincidencia indicará la existencia de algún defecto, por lo que habrá que proceder a detectar y corregir dicho error (mediante un proceso de depuración).



La complejidad ciclomática de McCabe es una métrica de la complejidad del módulo probado. Así, cuando $V(G)$ es mayor que 10, la probabilidad de defectos en el módulo o en el programa crece bastante si dicho valor alto no se debe a sentencias case o similares. En estos casos, es recomendable replantearse el diseño modular obtenido, de modo que sería conveniente dividir el módulo complejo en varios módulos para reducir la complejidad del software.

Actividad propuesta 3.1

Prueba del camino básico

El siguiente programa escrito en Java pide un número entero por teclado e indica si es un número perfecto, es decir, si es igual a la suma de sus divisores propios. Un divisor propio es un entero positivo distinto del número en sí mismo y que divide al número de forma exacta, es decir, sin resto.

Construye el grafo de flujo para este programa y calcula su complejidad ciclomática. Además, indica un conjunto de caminos independientes y, para cada camino, el caso de prueba correspondiente, incluyendo la entrada proporcionada y la salida esperada.

```
1 public static void main(String[] args) {
2     int num, divisor, sumadivisores;
3     divisor = 1;
4     sumadivisores = 0;
5     Scanner entrada = new Scanner(System.in);
6     System.out.print ("Introduzca un número mayor que 0: ");
7     num = entrada.nextInt();
8     while (divisor <= num/2)
9     {
10         if (num % divisor == 0)
11             sumadivisores = sumadivisores + divisor;
12         divisor++;
13     }
14     if (num == sumadivisores)
15         System.out.println ("El número " + num + " es un número perfecto");
16     else
17         System.out.println ("El número " + num + " no es un número perfecto");
18 }
```